

Natural Computing

Definition Sheet

Thomas Gabor
LMU Munich

Maximilian Zorn
LMU Munich

summer term 2025

Notation. $\mathbb{B}$ is the set of truth values or Booleans, i.e., $\mathbb{B} = \{0, 1\}$. $\mathbb{N}$ is the set of natural numbers starting from zero, i.e., $\mathbb{N} = \{0, 1, 2, \dots\}$ so that it holds that $\mathbb{B} \subset \mathbb{N} \subset \mathbb{Z} \subset \mathbb{R} \subset \mathbb{C}$. $\mathbb{P}$ denotes the space of probabilities and $\mathbb{F}$ denotes the space of fuzzy values with $\mathbb{P} = \mathbb{F} = [0; 1] \subset \mathbb{R}$ being only discerned for semantic but not mathematical reasons. $\mathbb{E}$ denotes the expected value.

$\wp(X) = 2^X$ denotes the power set of X . $\uplus$ denotes vector or sequence concatenation, i.e., given two vectors $\mathbf{x} = \langle x_1, \dots, x_{|\mathbf{x}|} \rangle$ and $\mathbf{y} = \langle y_1, \dots, y_{|\mathbf{y}|} \rangle$, $\mathbf{x} \uplus \mathbf{y} = \langle x_1, \dots, x_{|\mathbf{x}|}, y_1, \dots, y_{|\mathbf{y}|} \rangle$. A vector $\langle x_0, \dots, x_{n-1} \rangle$ with length $n \in \mathbb{N}$ can also be written as $\langle x_i \rangle_{0 \leq i < n}$ for a new iteration variable i . A sequence containing an arbitrary amount of elements from the space X has the type $\langle X \rangle$. $_$ denotes unspecified function arguments ($f(_) = 0$ is the constant function that always returns zero, e.g.). For any finite set $X = \{x_0, \dots, x_n\}$, $|X| = n$ denotes the number of elements in X . For infinite sets, $|X| = \infty$.

Definition 1 (cellular automaton). Let $G = (V, E)$ be a graph with vertices V and edges $E \subseteq V \times V$. Let $neighborhood : V \rightarrow V^{d+1}$ for some neighborhood degree $d \in \mathbb{N}$ be a function that returns an ordered vector of neighbors of a given node v , always including v itself. A state $x \in \mathcal{X}$ is a mapping of vertices to the values $\{0, 1\}$, i.e., the state space $\mathcal{X}$ is given via $\mathcal{X} = (V \rightarrow \{0, 1\})$. Let x_t be a state that exists at time step $t \in \mathbb{N}$. The evolution of a state x_t to its subsequent state x_{t+1} is given deterministically via a function $f : \{0, 1\}^{d+1} \rightarrow \{0, 1\}$ so that

$$x_{t+1}(v) = f(x_t(u_1), \dots, x_t(u_{d+1}))$$

where $\vec{u} = \langle u_1, \dots, u_{d+1} \rangle = neighborhood(v)$.

A tuple (G, f, x_0) is called a cellular automaton with initial state $x_0 \in \mathcal{X}$.

Definition 2 (1D cellular automaton). A cellular automaton (G, f, x_0) is called a 1D cellular automaton iff all vertices have exactly one incoming edge (“left neighbor”) and one outgoing edge (“right neighbor”) and $neighborhood(v) = \langle u, v, w \rangle$ where $(u, v) \in E$ and $(v, w) \in E$ and thus $d = 2$. Subsequently, we usually write

$$x_{t+1}(v) = f(x_t(u), x_t(v), x_t(w))$$

for the evolution where u is the left neighbor of v and w is the right neighbor of v .

Definition 3 (Conway’s game of life (standard)). Let $G = (V, E)$ be a grid graph with vertices V and (undirected) edges $E \subseteq V \times V$ with a fixed degree of 8 for all nodes. We define $surroundings : V \rightarrow \wp(V)$ via

$$surroundings(v) = \{w \mid (v, w) \in E\}$$

so that $|surroundings(v)| = 8$ and $v \notin surroundings(v)$ for all $v \in V$. A state $x \in \mathcal{X}$ is a mapping of vertices to the labels $\{dead, alive\}$, i.e., the state space $\mathcal{X}$ is given via $\mathcal{X} = (V \rightarrow \{dead, alive\})$. Let x_t be a state that exists at time step $t \in \mathbb{N}$. We define

$$|v|_{x_t} = |\{w \mid w \in surroundings(v) \wedge x_t(w) = alive\}|.$$

In the game of life, the evolution of a state x_t to its subsequent state x_{t+1} is given deterministically via

$$x_{t+1}(v) = \begin{cases} dead & \text{if } |v|_{x_t} \leq 1, \\ x_t(v) & \text{if } |v|_{x_t} = 2, \\ alive & \text{if } |v|_{x_t} = 3, \\ dead & \text{if } |v|_{x_t} \geq 4, \end{cases}$$

for all $v \in V$. A tuple (G, x_0) is called an instance of the game of life for initial state $x_0 \in \mathcal{X}$.

Definition 4 (Turing machine [3]). A Turing machine is a tuple $(Q, \Sigma, \Gamma, \sqcup, q_S, q_F, \delta)$ so that

- Q is a finite set of states,
- Σ is a finite input/output alphabet (e.g., $\Sigma = \{0, 1\}$),
- Γ is a finite tape alphabet with $\Sigma \subset \Gamma$,
- $\sqcup \in \Gamma$ is a blank symbol with $\sqcup \in \Sigma$,
- $q_S \in Q$ is a start state,
- $q_F \in Q$ is a stop state,
- and $\delta : Q \setminus \{q_F\} \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, N\}$ is a partially defined transition function.

Theorem 1 (Church-Turing thesis). Any computable function can be computed by a Turing machine.

Theorem 2 (extended Church-Turing thesis). Any computable function can be computed by a Turing machine *with at most polynomial overhead*.

Definition 5 (quantum state). Let $\mathcal{X}$ be an arbitrary state space. Let $\mathcal{Q}(\mathcal{X})$ be the space of superposition states with bases in $\mathcal{X}$, i.e., each $q \in \mathcal{Q}(\mathcal{X})$ has the form

$$q = \sum_{x \in \mathcal{X}} c_x |x\rangle$$

where $c_x \in \mathbb{C}$ for any $x \in \mathcal{X}$ is called an amplitude so that

$$\sum_{x \in \mathcal{X}} |c_x|^2 = 1$$

and $|x\rangle$ (read “ket x ”) denotes a vector corresponding to the state $x \in \mathcal{X}$.

We use the following terminology:

- If for more than one $x \in \mathcal{X}$ it holds that $c_x \neq 0$, then q is called a *superposition* state.
- If $\mathcal{X} = \mathcal{X}_A \times \mathcal{X}_B$, i.e., $\mathcal{X}$ can be constructed as a product of two other state spaces $\mathcal{X}_A, \mathcal{X}_B$, then it follows that q has the form

$$q = \sum_{\substack{x_A \in \mathcal{X}_A, \\ x_B \in \mathcal{X}_B}} c_{x_A, x_B} |x_A\rangle \otimes |x_B\rangle$$

where $\otimes$ is the tensor product. If q cannot be written as

$$q = \left(\sum_{x_A \in \mathcal{X}_A} c_{x_A} |x_A\rangle \right) \otimes \left(\sum_{x_B \in \mathcal{X}_B} c_{x_B} |x_B\rangle \right),$$

then we call q an *entangled* state.

- If $\mathcal{X} = \{0, 1\}$ (i.e., any $x \in \mathcal{X}$ is a bit), then any $q \in \mathcal{Q}(\mathcal{X})$ is called a *qubit* and can be written as

$$q = c_0 |0\rangle + c_1 |1\rangle$$

for $c_0, c_1 \in \mathbb{C}$ with $|c_0|^2 + |c_1|^2 = 1$.

- If $\mathcal{X} = \{0, 1\}^n$ for any $n \in \mathbb{N}$ (i.e., any $x \in \mathcal{X}$ is a bit register), then any $q \in \mathcal{Q}(\mathcal{X})$ is called a *qubit register* of size n and can be written as

$$q = \sum_{i=0}^{2^n-1} c_i |i\rangle = c_{0\dots 0} |0\dots 0\rangle + \dots + c_{1\dots 1} |1\dots 1\rangle$$

where we usually denote i in binary as illustrated above.

- When we *measure* a quantum state $q \in \mathcal{Q}(\mathcal{X})$, we generate classical state $M(q) \in \mathcal{X}$ so that for all $x \in \mathcal{X}$ it holds that

$$\Pr(M(q) = x) = |c_x|^2.$$

Definition 6 (quantum gate). Let $q = \sum_{i=0}^{2^n-1} c_i |i\rangle$ be an n -qubit register. We often write

$$q = \begin{bmatrix} c_{0\dots 0} \\ \vdots \\ c_{1\dots 1} \end{bmatrix}$$

as a shorter notation when the basis $|0\dots 0\rangle, \dots, |1\dots 1\rangle$ is fixed.

An n -ary quantum gate is an operator that works on an n -qubit register q , i.e., (possibly) alters its superposition amplitudes, and thus produces a new n -qubit register q' . An n -ary quantum gate can be given via a matrix $U \in \mathbb{C}^{2^n \times 2^n}$ so that

$$q' = Uq.$$

Quantum gates commonly implemented on quantum computers are unitary, which — among other things — implies they are reversible.

Definition 7 (Ising model). Let $G = (V, E)$ be a graph with vertices $V = \{v_0, \dots, v_{n-1}\}$ for $n \in \mathbb{N}$ and undirected edges $E \subseteq V \times V$. A spin configuration $x \in \mathcal{X} = (V \rightarrow \{-1, +1\})$ is a mapping of vertices to values $\{-1, +1\}$. This mapping x is fully defined by a spin vector $\vec{\sigma} \in \{-1, +1\}^n$ so that $x(v_i) = \sigma_i$ for any $i \in \{0, \dots, n-1\}$. The energy of an Ising model is then given via

$$H(\vec{\sigma}) = - \sum_{\substack{i \in \{0, \dots, n-1\} \\ j \in \{0, \dots, n-1\}}} J_{ij} \sigma_i \sigma_j - \sum_{i \in \{0, \dots, n-1\}} h_i \sigma_i$$

where $J \in \mathbb{R}^{n \times n}$ is called the interaction matrix and $\vec{h} \in \mathbb{R}^n$ is called the vector of field strengths. We also assume that for any $i, j \in \{0, \dots, n-1\}$ it holds that $i \geq j \implies J_{ij} = 0$, i.e., J is a strictly upper triangle matrix, and that $(v_i, v_j) \notin E \implies J_{ij} = 0$, i.e., J_{ij} can only have non-zero values where an edge between i and j exists in the substrate graph G .

For an Ising model given by (G, J, h) , there are processes (both artificial and natural) that produce a spin configuration $\vec{\sigma}$ so that $H(\vec{\sigma})$ is minimized.

Definition 8 (QUBO). Let $Q \in \mathbb{R}^{n \times n}$ be an arbitrary matrix called QUBO matrix. The process of finding a vector $\vec{x} \in \mathbb{B}^n$ so to

$$\text{minimize } H(\vec{x}) = x^\top Q x = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} Q_{ij} x_i x_j$$

is called *quadratic unconstrained binary optimization (QUBO)*. W.l.o.g. we can assume that Q is a (non-strictly) upper triangular matrix.

Definition 9 (optimization). Let $\mathcal{X}$ be an arbitrary state space. Let $\mathcal{T}$ be an arbitrary set called target space and let $\leq$ be a total order on $\mathcal{T}$. A total function $\tau : \mathcal{X} \rightarrow \mathcal{T}$ is called target function. Optimization (minimization/maximization) is the procedure of searching for an $x \in \mathcal{X}$ so that $\tau(x)$ is optimal (minimal/maximal). Unless stated otherwise, we assume minimization. An optimization run of length $g + 1$ is a sequence of states $\langle x_t \rangle_{0 \leq t \leq g}$ with $x_t \in \mathcal{X}$ for all t .

We use the following terminology:

- An optimization run is *working to some extent* if $t \leq t'$ for $t, t' \in [1; g] \subset \mathbb{N}$ correlates positively to some extent with $\tau(x_t) \geq \tau(x_{t'})$.
- An optimization run is *improving* if $\tau(x_t) \geq \tau(x_{t'})$ for all $t \leq t'$.
- An optimization run has *found a global optimum* iff $\tau(x) \geq \tau(x_g)$ for all $x \in \mathcal{X}$.

Let $e : \langle \mathcal{X} \rangle \times (\mathcal{X} \rightarrow \mathcal{T}) \rightarrow \mathcal{X}$ be a possibly randomized or non-deterministic function so that the optimization run $\langle x_t \rangle_{0 \leq t \leq g}$ is produced by calling e repeatedly, i.e., $x_{t+1} = e(\langle x_u \rangle_{0 \leq u \leq t}, \tau)$ for all t , $1 \leq t \leq g$, where x_0 is given externally (e.g., $x_0 =_{def} 42$) or chosen randomly (e.g., $x_0 \sim \mathcal{X}$). An optimization process is a tuple $(\mathcal{X}, \mathcal{T}, \tau, e, \langle x_t \rangle_{0 \leq t \leq g})$.

We use the following terminology:

- The *end result* of an optimization process $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_t \rangle_{0 \leq t \leq g})$ is x_g .
- The *best result* of an optimization process $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_t \rangle_{0 \leq t \leq g})$ is

$$*\mathcal{D} = \arg \min_{\substack{x_t \\ 0 \leq t \leq g}} \tau(x_t)$$

with the *best target value* $|\mathcal{D}| = \tau(*\mathcal{D})$.

- An optimization process $\mathcal{D}' = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x'_t \rangle_{0 \leq t \leq g'})$ is a *continuation* of an optimization process $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_t \rangle_{0 \leq t \leq g})$ iff $g' \geq g$ and $x'_t = x_t$ for all t with $0 \leq t \leq g$. We write $\ell(\mathcal{D})$ for the set of all possible continuations of $\mathcal{D}$, which can be large if e is randomized or non-deterministic.
- An optimization process $\mathcal{D}$ is *elitist* iff for all its continuations $\mathcal{D}' = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x'_t \rangle_{0 \leq t \leq g'}) \in \ell(\mathcal{D})$ it holds that $\tau(x'_t) \geq \tau(x'_{t'})$ for all $t \leq t'$.

Algorithm 1 (single-sample random search). Let $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$ be an optimization process. The process $\mathcal{D}$ continues via (single-sample) random search if e is of the form

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \arg \min_{x \in \{x_t, x'_t\}} \tau(x)$$

where $x'_t \sim \mathcal{X}$ is drawn at random for each call to e .

Algorithm 2 (stochastic hill climbing). Let $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$ be an optimization process. Let $neighbors : \mathcal{X} \rightarrow \wp(\mathcal{X})$ be a function that returns a set of neighbors of a given state $x \in \mathcal{X}$. The process $\mathcal{D}$ continues via stochastic hill climbing if e is of the form

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \arg \min_{x \in \{x_t, x'_t\}} \tau(x)$$

where $x'_t \sim neighbors(x_t)$ is drawn at random for each call to e .

Algorithm 3 (simulated annealing). Let $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$ be an optimization process. Let $neighbors : \mathcal{X} \rightarrow \wp(\mathcal{X})$ be a function that returns a set of neighbors of a given state $x \in \mathcal{X}$. Let $\kappa : \mathbb{N} \rightarrow \mathbb{R}$ be a temperature schedule, i.e., a function that returns a temperature value for each time step. Let $A : \mathcal{T} \times \mathcal{T} \times \mathbb{R} \rightarrow \mathbb{P}$ with $\mathbb{P} = [0; 1] \subset \mathbb{R}$ be a function that returns an acceptance probability given two target values and a temperature. Commonly, we use

$$A(Q, Q', K) = e^{-\frac{(Q' - Q)}{K}}$$

for $\mathcal{T} \subseteq \mathbb{R}$ and Euler's number e . The process $\mathcal{D}$ continues via simulated annealing if e is of the form

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \begin{cases} x'_t & \text{if } \tau(x'_t) \leq \tau(x_t) \text{ or } r \leq A(\tau(x_t), \tau(x'_t), \kappa(t)), \\ x_t & \text{otherwise,} \end{cases}$$

where $x'_t \sim neighbors(x_t)$ and $r \sim \mathbb{P}$ are drawn at random for each call to e .

Algorithm 4 (gradient descent). Let $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e, \langle x_u \rangle_{0 \leq u \leq t})$ be an optimization process. Let $\mathcal{T}$ be continuous ($\mathcal{T} = \mathbb{R}$, e.g.) and let $\tau' : \mathcal{X} \rightarrow \mathcal{T}$ be the first derivative of τ . The process $\mathcal{D}$ continues via gradient descent (with update rate $\alpha \in \mathbb{R}^+$) if e is of the form

$$e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = x_t - \alpha \cdot \tau'(x_t).$$

The learning rate α can also be given as a function, usually $\alpha : \mathbb{N} \rightarrow \mathbb{R}$ so that $e(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = x_t - \alpha(t) \cdot \tau'(x_t)$. If τ is stochastic, this process is called stochastic gradient descent (SGD).

Algorithm 5 (simulated annealing with local gradient descent). For $n \in \mathbb{N}$, $\mathcal{D} = (\mathcal{X}, \mathcal{T}, \tau, e_{\mathcal{D}}, \langle x_u \rangle_{0 \leq u \leq t})$ is an optimization process that continues via simulated annealing with n steps of local gradient descent if $e_{\mathcal{D}}$ has the form

$$e_{\mathcal{D}}(\langle x_u \rangle_{0 \leq u \leq t}, \tau) = x_{t+1} = \begin{cases} {}^*\mathcal{E}(x'_t) & \text{if } \tau({}^*\mathcal{E}(x'_t)) \leq \tau(x_t) \\ & \text{or } r \leq A(\tau(x_t), \tau({}^*\mathcal{E}(x'_t)), \kappa(t)), \\ x_t & \text{otherwise,} \end{cases}$$

where x'_t, r, A, κ are given as in Alg. 3 and where $\mathcal{E}(x'_t) = (\mathcal{X}, \mathcal{T}, \tau, e_{\mathcal{E}}, \langle x'_{t+v} \rangle_{0 \leq v \leq n})$ is an optimization process that continues via gradient descent (cf. Alg. 4) so that

$${}^*\mathcal{E}(x'_t) = \arg \min_{\substack{x'_{t+v}, \\ 0 \leq v \leq n}} \tau(x'_{t+v})$$

is the best solution found by $\mathcal{E}$ when starting at x'_t .

Theorem 3 (no free lunch [1, 5]). As measured by sample efficiency, i.e., the achieved minimal value of τ per evaluations of $\tau(x)$ for some new $x \in \mathcal{X}$ for finite $\mathcal{X}$, all optimization algorithms perform the same when averaged over all possible target functions τ . Thus, for any search/optimization algorithm, any elevated performance over one class of problems is exactly paid for in performance over another class.

Definition 10 (artificial chemistry, soup). Let $\mathcal{X}$ be a state space. Let $\mathcal{R}$ be a set of reaction rules $R \in \mathcal{R}$ where each R is a function $R : \mathcal{X}_R^{\text{dom}} \rightarrow \mathcal{X}_R^{\text{cod}}$ for $\mathcal{X}_R^{\text{dom}}, \mathcal{X}_R^{\text{cod}} \subseteq \wp^*(\mathcal{X})$. Let $\mathcal{A}(\mathcal{X}, \mathcal{R})$ be the set of all *reaction materials* from $X \in \wp^*(\mathcal{X})$ for all rules in $\mathcal{R}$, i.e.,

$$\mathcal{A}(\mathcal{X}, \mathcal{R}) = \{(X', R) \mid X' \subseteq X \wedge X' \in \mathcal{X}_R^{\text{dom}} \wedge R \in \mathcal{R}\}.$$

Let $A : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X}) \times \mathcal{R}$ be a possibly randomized or non-deterministic function that takes a multi-set $X \in \wp^*(\mathcal{X})$ and returns a multi-set of reactants and a reaction rule to perform so that $A(X) \in \mathcal{A}(\mathcal{X}, \mathcal{R})$. The tuple $(\mathcal{X}, \mathcal{R}, A)$ defines an artificial chemistry. A multi-set $X \in \wp^*(\mathcal{X})$ of an artificial chemistry is also called soup; each element $x \in X$ is also called a particle. The evolution of a soup for g generations is a sequence $\langle X_0, \dots, X_g \rangle$ so that X_0 is given as the initial soup and, for all t with $0 \leq t \leq g-1$

$$X_{t+1} = (X_t \setminus X') \uplus R(X')$$

where $(X', R) = A(X_t)$.

Reaction rules are usually written in the form $R = \sum_i x_i \rightarrow \sum_j y_j$ for $x_i, y_j \in \mathcal{X}$, $i, j \in \mathbb{N}$, $\{x_i\}_i \in \mathcal{X}_R^{\text{dom}}$, $\{y_j\}_j \in \mathcal{X}_R^{\text{cod}}$, and $R(\{x_i\}_i) = \{y_j\}_j$.

Definition 11 (population-based optimization). Let $\mathcal{X}$ be a state space. Let $\mathcal{T}$ be a target space with total order $\leq$. Let $\tau : \mathcal{X} \rightarrow \mathcal{T}$ be a target function. A tuple $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \tau, E, \langle X_t \rangle_{0 \leq t \leq g})$ is a population-based optimization process iff $X_t \in \wp^*(\mathcal{X})$ for all t and $\bar{E} : \langle \wp^*(\mathcal{X}) \rangle \times (\mathcal{X} \rightarrow \mathcal{T}) \rightarrow \wp^*(\mathcal{X})$ is a possibly randomized, non-deterministic, or further parametrized function so that the population-based optimization run is produced by calling E repeatedly, i.e., $X_{t+1} = E(\langle X_u \rangle_{0 \leq u \leq t}, \tau)$ where X_0 is given externally or chosen randomly.

Definition 12 (population-based optimization (alternate)). An optimization process $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \tau, E, \langle X_t \rangle_{0 \leq t \leq g})$ is called population-based iff $\mathcal{X}$ has the form $\mathcal{X} = \wp^*(\mathcal{Y})$ for some other state space $\mathcal{Y}$.

Algorithm 6 (basic evolutionary algorithm). Let $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \tau, E, \langle X_u \rangle_{0 \leq u \leq t})$ be a population-based optimization process. The process $\mathcal{E}$ continues via an evolutionary algorithm if E has the form

$$E(\langle X_u \rangle_{0 \leq u \leq t}, \tau) = X_{t+1} = \textit{selection}(X_t \uplus \textit{variation}(X_t))$$

where *selection* and *variation* are possibly randomized or non-deterministic functions so that for any $X \in \wp^*(\mathcal{X})$ it holds that $|\textit{selection}(X)| \leq |X|$ and $|\textit{selection}(X \uplus \textit{variation}(X))| = |X|$.

Algorithm 7 (typical evolutionary algorithm). Let $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \phi, E, \langle X_u \rangle_{0 \leq u \leq t})$ be a population-based optimization process.

- Let $\sigma_N^{survivors}, \sigma_N^{parents} : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X})$ be (possibly randomized or non-deterministic) selection functions that return $N \in \mathbb{N}$ individuals, i.e., $|\sigma_N(X)| = N$ and $\sigma_N(X) \subseteq X$ for all X . They may possibly depend on $\mathcal{E}$ (most commonly ϕ). Let $\varrho_N^{mutants} : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X})$ be a special selection function that, given a population X , returns a random subset $X' \subseteq X$ so that $|X'| = N$. Note that $\varrho_{|X|}(X) = X$ for all X .
- Let $mutate : \mathcal{X} \rightarrow \mathcal{X}$ be a (possibly randomized or non-deterministic) single-individual mutation function. We write $mutate[\cdot] : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X})$ for the per-individual application of $mutate$, i.e., $mutate[X] = \{mutate(x) : x \in X\}$.
- Let $recombine : \mathcal{X} \times \mathcal{X} \rightarrow \mathcal{X}$ be a (possibly randomized or non-deterministic) two-parent recombination function. We write $recombine[\cdot] : \wp^*(\mathcal{X}) \rightarrow \wp^*(\mathcal{X})$ for the random-pairing application of $recombine$, i.e., $recombine[X] = \{recombine(x_1, x_2)\} \uplus recombine[X \setminus \{x_1, x_2\}]$ with $x_1, x_2 \sim X$ and $recombine[\{x\}] = \emptyset$ for all x as well as $recombine[\emptyset] = \emptyset$.
- Let $migrate : \emptyset \rightarrow \mathcal{X}$ be a (possibly randomized or non-deterministic) individual creation function. We write $migrate_N : \emptyset \rightarrow \wp^*(\mathcal{X})$ with $N \in \mathbb{N}$ for the iterated application of $migrate$, i.e., $migrate_N[\cdot] = \{migrate()\} \uplus migrate_{N-1}[\cdot]$ with $migrate_0[\cdot] = \emptyset$.

The process $\mathcal{E}$ continues via a typical evolutionary algorithm if E has the form

$$\begin{aligned} E(\langle X_u \rangle_{0 \leq u \leq t}, \phi) = X_{t+1} = & \sigma_{|X_t|}^{survivors}(X_t) \\ & \uplus mutate[\varrho_M^{mutants}(X_t)] \\ & \uplus recombine[\sigma_{2C}^{parents}(X_t)] \\ & \uplus migrate_H[\cdot] \end{aligned}$$

where $M \in \mathbb{N}$ is the number of mutants produced per generation, $C \in \mathbb{N}$ is the number of children produced per generation, and $H \in \mathbb{N}$ is the number of migrants (sometimes called hypermutants) generated per generation. Especially M is often expressed as a rate for random choice within the population, i.e., $M = \lceil m \cdot |X_t| \rceil$ where $m \in \mathbb{R}$ is called mutation rate.

Algorithm 8 (diversity-aware evolutionary algorithm). Let $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \phi, E, \langle X_u \rangle_{0 \leq u \leq t})$ be the process of an evolutionary algorithm (cf. Alg. 7). $\mathcal{E}$ is called a diversity-aware evolutionary algorithm if the used fitness function $\phi : \mathcal{X} \rightarrow \mathcal{T}$ has the form

$$\phi(x) = (1 - \zeta) \cdot \tau(x) + \zeta \cdot \delta(x)$$

where $\tau : \mathcal{X} \rightarrow \mathcal{T}$ is the underlying target function of the given optimization problem, $\delta : \mathcal{X} \rightarrow \mathcal{T}$ is a (possibly randomized, non-deterministic, or further parametrized) function that estimates the diversity that a given individual contributes to the population, and $\zeta \in [0; 1] \subset \mathbb{R}$ is called a diversity weight. Note that according to Thm. 4 and Gabor et al. [2] the diversity estimate

$$\delta(x) = \frac{1}{|X_t|} \cdot \sum_{x' \in X_t} |x' - x|$$

is generally suitable for $\mathcal{X} = \mathbb{R}^n$ or $\mathcal{X} = \mathbb{Z}^n$ for some n .

Theorem 4 (Wineberg-Oppacher diversity [4]). All measures of the diversity of an individual x in a population X are related to the pairwise distance between individuals in X as measured by a suitable norm (Euclidean for real-valued representations, Hamming for symbolic representations, e.g.).

Algorithm 9 (evolutionary algorithm with self-adaptive mutation rate). Let $\mathcal{E} = (\mathcal{X}, \mathcal{T}, \phi, E, \langle X_u \rangle_{0 \leq u \leq t})$ be the process of an evolutionary algorithm (cf. Alg. 7). $\mathcal{E}' = (\mathcal{X}', \mathcal{T}, \phi', E', \langle X'_u \rangle_{0 \leq u \leq t})$ is called an evolutionary algorithm with self-adaptive mutation rate if

- $\mathcal{X}' = \mathcal{X} \times \mathbb{R}$,
- $\phi'((x, m)) = \phi(x)$,
- a new meta-mutation weight $W \in \mathbb{R}$ is given alongside a function $rate-mutate : \mathbb{R} \rightarrow \mathbb{R}$ that has the form

$$rate-mutate(m) = cap(m + W \cdot (r - 0.5))$$

where cap maps all inputs beyond $[0; 1]$ to their nearest point within $[0; 1]$ (i.e., 0 or 1) and $r \sim [0; 1] \subseteq \mathbb{R}$ is a number drawn at random,

- E' uses a new mutation function $mutate'$ that has the form

$$mutate'((x, m)) = \begin{cases} (mutate(x), rate-mutate(m)) & \text{if } r \leq m, \\ (x, m) & \text{otherwise,} \end{cases}$$

where $mutate$ is the mutation function of E and $r \sim [0; 1] \subseteq \mathbb{R}$ is a number drawn at random,

- all individuals are subjected to the $mutate'$ function, i.e., $M = |X|$,
- E' uses a new recombination function $recombine'$ that has the form

$$recombine'((x_1, m_1), (x_2, m_2)) = \begin{cases} (recombine(x_1, x_2), m_1) & \text{if } r \leq 0.5, \\ (recombine(x_1, x_2), m_2) & \text{otherwise,} \end{cases}$$

where $recombine$ is the recombination function of E and $r \sim [0; 1] \subseteq \mathbb{R}$ is a number drawn at random,

- E' uses a new migration function $migrate'$ that has the form

$$migrate'() = (migrate(), r)$$

where $migrate$ is the migration function of E and $r \sim [0; 1] \subseteq \mathbb{R}$ is a number drawn at random.

References

- [1] No free lunch theorems. <http://www.no-free-lunch.org>. Accessed 2022-05-10.
- [2] Thomas Gabor, Lenz Belzner, and Claudia Linnhoff-Popien. Inheritance-based diversity measures for explicit convergence control in evolutionary algorithms. In *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 841–848, 2018.
- [3] Encyclopedia of Mathematics. Turing machine. http://encyclopediaofmath.org/index.php?title=Turing_machine&oldid=31220. Accessed 2022-04-26.
- [4] Mark Wineberg and Franz Oppacher. The underlying similarity of diversity measures used in evolutionary computation. In *Genetic and evolutionary computation conference*, pages 1493–1504. Springer, 2003.
- [5] David H Wolpert and William G Macready. No free lunch theorems for optimization. *IEEE transactions on evolutionary computation*, 1(1):67–82, 1997.